Modification in phase1

We will replace IAM - - > RBAC

Load balancer - - > ngnix

Arch diagram - - >Modified one

EC2 instances - - > 3-6 instances

S3 - - > from 1 to 4 S3

RDS - - > from 1 to 5

Kafka clusters - - >from 2 to (5 for zookeeper and 5 broker)

Team members roles

1. Member 1: Team Lead & Platform Engineer

Role: The "Traffic Controller" **Why:** Without you, the services exist in isolation. You provide the roads (Kafka) and the front door (API Gateway).

- **Task 1: Kafka Cluster (Sec 3.1)**
  - Deploy a **3-node Kafka Cluster** and a **3-node Zookeeper Ensemble** on your EC2 instances.
  - **Critical:** You must configure the **Topics** that the team needs:
    - `document.uploaded`, `document.processed`, `notes.generated`
    - `quiz.requested`, `quiz.generated`
    - `audio.transcription.requested`, `audio.transcription.completed`
    - `audio.generation.requested`, `audio.generation.completed`
    - `chat.message`.
- **Task 2: API Gateway (Sec 4.1)**
  - Set up the single entry point.
  - **Routes:** You must route traffic like `/api/tts/*` to Member 2's service, `/api/stt/*` to Member 3's service, etc..

Integration:

- Work with all 5 members to connect their services to the API Gateway.
- Configure the Kafka topics required by the team.
- Share the Kafka cluster address (e.g., internal NLB DNS) with the team so they can configure their environment variables.

---

2. Member 2: Text-to-Speech (TTS) Service Owner

Role: The "Voice Generator" **Why:** To allow users to convert their text notes into audio for listening on the go.

- **APIs to Build:**
  - `POST /api/tts/synthesize` (Input text, start job).
  - `GET /api/tts/audio/{id}` (Download the MP3).
- **Kafka Work:**
  - **Listen (Consume):** `audio.generation.requested`.
  - **Speak (Produce):** `audio.generation.completed`.
- **Tech Stack:** Python 3.11, **gTTS** or **AWS Polly SDK**.
- **Storage:** Save audio files to your bucket `tts-service-storage-{env}`.

---

3. Member 3: Speech-to-Text (STT) Service Owner

Role: The "Transcriber" **Why:** To allow users to upload lecture recordings and get text notes back.

- **APIs to Build:**
  - `POST /api/stt/transcribe` (Upload audio file).
  - `GET /api/stt/transcription/{id}` (Get the text).
- **Kafka Work:**
  - **Listen (Consume):** `audio.transcription.requested`.
  - **Speak (Produce):** `audio.transcription.completed`.
- **Tech Stack:** Python 3.11, **Whisper** (recommended) or AWS Transcribe.
- **Storage:** Save uploads to `stt-service-storage-{env}`.

---

4. Member 4: Chat Completion Service Owner

Role: The "AI Tutor" **Why:** To let students ask questions about their documents ("What did this PDF say about biology?").

- **APIs to Build:**
  - POST /api/chat/message (Send user question).
  - `GET /api/chat/conversations` (List history).
- **Kafka Work:**
  - **Listen (Consume):** `document.processed` (Crucial! This is how you learn what is in the documents to answer questions later).
  - **Speak (Produce):** `chat.message`.
- **Tech Stack:** Python 3.11, **LangChain** or **OpenAI SDK**.
- **Storage:** Store chat history in `chat-service-storage-{env}` and PostgreSQL.

---

5. Member 5: Document Reader Service Owner

Role: The "Ingestion Engine" **Why:** This is the starting point. Users upload files here, and this service extracts the text for everyone else to use.

- **APIs to Build:**
  - `POST /api/documents/upload`.
  - `GET /api/documents/{id}/notes` (AI summary of the file).
- **Kafka Work:**
  - **Speak (Produce):** `document.uploaded`, `document.processed`, and `notes.generated`.
  - **Listen (Consume): NONE**. You trigger the chain; you don't wait for anyone.

- **Tech Stack:** Python 3.11, **PyPDF2**, **python-docx**.
- **Storage:** Save PDFs to `document-reader-storage-{env}`.

---

6. Member 6: Quiz Service Owner (and Security)

Role: The "Examiner" **Why:** To automatically generate quizzes from the uploaded documents so students can test their knowledge.

- **APIs to Build:**
  - `POST /api/quiz/generate`.
  - `POST /api/quiz/{id}/submit`.
- **Kafka Work:**
  - **Listen (Consume):** `quiz.requested` and `notes.generated` (You need the notes to make the quiz questions).
  - **Speak (Produce):** quiz.generated.
- **Tech Stack:** Python 3.11, **OpenAI SDK**, **LangChain**.
- **Storage:** Save quizzes to `quiz-service-storage-{env}` and PostgreSQL.

The Final Result of Phase 2

By December 4th, you will have:

1. **A Working Nervous System:** A Kafka cluster that actually passes messages.
2. **5 "Brain" Containers:** Python microservices that can do their specific jobs (Read, Speak, Listen, Chat, Quiz).
3. **Communication:** When Member 5 processes a PDF, Member 6 will automatically know about it via Kafka and can be ready to make a quiz.
4. **No Direct Database Access:** You will have proven you can build "Shared Nothing" architecture, which is a high-level cloud skill.

Phase 2: Platform Engineering Documentation

Role: Member 1 (Integration Engineer & Team Lead)

Goal: Build the "Roads" (Network) and "Traffic Signs" (Gateway) so the team can work.

---

1. Role Overview

As Member 1, I do not write the Python applications (like the Chat or Quiz bots). Instead, I am the **Integration Engineer**.

- **The Switchboard:** I build the Nginx Gateway to route traffic from the internet to my team's private servers.
- **The Nervous System:** I set up the Kafka Cluster so the different parts of the application can send messages to each other.

---

2. Part 1: The Infrastructure (Terraform)

Your first task is to build the cloud environment using the provided Terraform script (`Phase2ScriptMain.txt`).

What You Are Building

1. **Network (VPC):** A private network for your team called "Project-Hydra-VPC".
2. **Public Zone:** One specific area (Subnet) accessible from the internet. This is where **Nginx** lives.
3. **Private Zone:** A hidden area for your team's code. This contains **3 Nodes** (servers) where the applications and Kafka run.
4. **Gateway:** A NAT Gateway that allows the private nodes to download updates but keeps them safe from hackers.

Action Items

- Run the Terraform script.
- **Budget Warning:** This script creates 4 servers (1 Nginx + 3 Nodes). Always run `terraform destroy` when you are done working to save money.

---

3. Part 2: The "Front Door" (Nginx Configuration)

Your teammates' applications live on private servers. The outside world cannot reach them directly. You must configure Nginx to act as the "Front Door".

The Routing Contract

You need to edit the `nginx.conf.tpl` file. The provided template is missing some members. You must ensure **all** the following routes exist:

URL Path	Send Traffic To...	Service Owner
`/api/tts`	Node 1 (Port 5000)	Member 2 (Text-to-Speech)
`/api/stt`	Node 2 (Port 5000)	Member 3 (Speech-to-Text)
`/api/chat`	Node 3 (Port 5000)	Member 4 (Chat Bot)
`/api/documents`	**Node 1 (Port 5001)**	Member 5 (Document Reader)
`/api/quiz`	**Node 2 (Port 5001)**	Member 6 (Quiz Generator)

Important Note: Since you only have 3 nodes but 5 members, Members 5 and 6 must share nodes with others. You must assign them **Port 5001** and update your Nginx config to match.

---

4. Part 3: The "Nervous System" (Kafka Setup)

Your team is building an "Event-Driven Architecture." This means their apps don't talk directly; they send messages to Kafka.

You must log in to one of the private nodes and run these exact commands to create the topics. If you don't do this, their apps will crash.

1. Document Topics (For Member 5)

Bash
```
bin/kafka-topics.sh --create --topic document.uploaded --bootstrap-server
localhost:9092
bin/kafka-topics.sh --create --topic document.processed --bootstrap-server
localhost:9092
bin/kafka-topics.sh --create --topic notes.generated --bootstrap-server
localhost:9092
```

2. Quiz Topics (For Member 6)

Bash
```
bin/kafka-topics.sh --create --topic quiz.requested --bootstrap-server
localhost:9092
bin/kafka-topics.sh --create --topic quiz.generated --bootstrap-server
localhost:9092
```

3. Audio Topics (For Members 2 & 3)

Bash
```
bin/kafka-topics.sh --create --topic audio.generation.requested --bootstrap-
server localhost:9092
bin/kafka-topics.sh --create --topic audio.generation.completed --bootstrap-
server localhost:9092
bin/kafka-topics.sh --create --topic audio.transcription.requested --
bootstrap-server localhost:9092
bin/kafka-topics.sh --create --topic audio.transcription.completed --
bootstrap-server localhost:9092
```

4. Chat Topic (For Member 4)

Bash
```
bin/kafka-topics.sh --create --topic chat.message --bootstrap-server
localhost:9092
```

---

5. Deliverables to Your Team

Once the system is running, you must provide the following information to your team so they can configure their `.env` files:

1. **Nginx Public IP:** This is used for API access (e.g., how the frontend talks to the backend).
2. **Internal Node IPs:** These are the private IPs (e.g., `10.0.10.x`). Your team needs these to connect to Kafka.